

Complemento: Análise de Código + Arquitetura de Agentes PDCA Independentes

Crypto Analyzer Futures - Implementação Detalhada

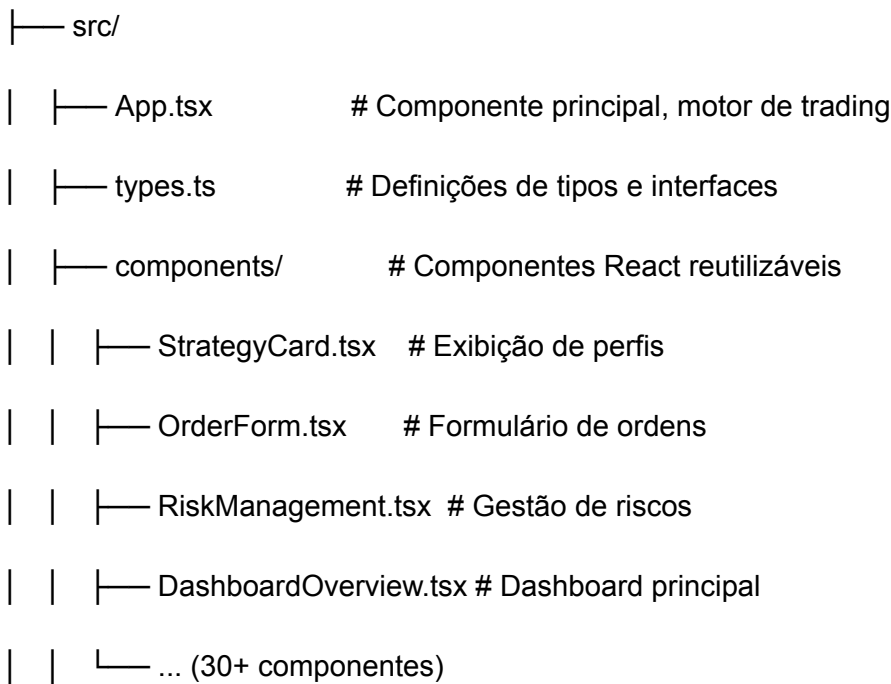
Data: 10 de Março de 2026
Versão: 2.0 - Análise Técnica Completa + Arquitetura de Agentes
Autor: Manus AI

1. Análise Detalhada do Código Existente

1.1 Estrutura do Projeto

O projeto Crypto Analyzer Futures é uma aplicação React 18 com TypeScript, hospedada no Vercel e integrada com Supabase para persistência de dados. A arquitetura segue o padrão de componentes React com serviços especializados.

Crypto-Analyzer-Futures/



```

|   └─ services/           # Serviços de negócio
|   |   └─ supabaseClient.ts # Cliente Supabase com proxy
|   |   └─ exchangeService.ts # Integração com corretoras
|   |   └─ syncService.ts    # Sincronização de dados
|   |   └─ marketService.ts  # Dados de mercado
|   |   └─ hyperliquidService.ts # Integração Hyperliquid
|   |   └─ auditService.ts   # Logs de auditoria
|   └─ utils/               # Utilitários
|   |   └─ technicalAnalysis.ts # Análise técnica
|   |   └─ calculations.ts    # Cálculos financeiros
|   |   └─ translations.ts    # Internacionalização
|   └─ contexts/           # React Contexts
|   |   └─ NotificationContext.tsx
|   └─ hooks/              # Custom React Hooks
|   |   └─ useMarketData.ts
|   └─ api/
|   |   └─ supabase.ts       # Proxy Vercel para Supabase
|   └─ scripts/
|       └─ worker.ts         # Worker para processamento

```

1.2 Estrutura de Dados Principais

StrategyProfile (Perfil de Investimento)

```
interface StrategyProfile {
```

id: string; // SAFE, MODERATE, BOLD, SPECIALIST, ALPHA

user_id?: string; // ID do usuário (Supabase)

name: string; // "Seguro", "Moderado", etc.

description: string; // Descrição curta

icon: string; // Nome do ícone (shield, scale, rocket, etc.)

color: string; // Cor para UI (blue, yellow, orange, purple, red)

riskLevel: string; // Low, Med, High, Expert, Extreme

confidenceThreshold: number; // 0-100: mínimo de confiança para trade

leverage: number; // 2x, 5x, 10x, 20x, 50x

capital: number; // Capital inicial alocado

currentCapital: number; // Capital atual (capital + PnL)

allocatedCapital: number; // Capital preso em posições abertas

marginPerTrade: number; // USD por trade

pnl: number; // Lucro/prejuízo acumulado

trades: number; // Total de trades

winRate: number; // Taxa de acerto (%)

active: boolean; // Se perfil está ativo

workflowSteps: string[]; // Etapas do workflow

stopLoss: number; // % de stop loss

takeProfit: number; // % de take profit

maxDrawdown?: number; // Máxima queda permitida (%)

```
indicators: AdvancedIndicators; // Configuração de indicadores

useDivergences: boolean;      // Usa divergências?

useCandlePatterns: boolean;   // Usa padrões de candle?

profileDailyTargetPct?: number; // Meta diária específica do perfil

profileDailyStopLossPct?: number; // Stop loss diário específico

}
```

AdvancedIndicators (Indicadores Técnicos)

```
interface AdvancedIndicators {

    rsi: IndicatorConfig;      // Relative Strength Index

    macd: IndicatorConfig;     // Moving Average Convergence Divergence

    stochastic: IndicatorConfig; // Stochastic Oscillator

    bollinger: IndicatorConfig; // Bollinger Bands

    ichimoku: IndicatorConfig; // Ichimoku Cloud

    sar: IndicatorConfig;      // Parabolic SAR

    cci: IndicatorConfig;      // Commodity Channel Index

    volume: IndicatorConfig;   // Volume Analysis

}
```

```
interface IndicatorConfig {

    enabled: boolean;          // Indicador ativo?

    period?: number;           // Período (ex: 14 para RSI)

    thresholdLow?: number;     // Limite baixo (oversold)

    thresholdHigh?: number;    // Limite alto (overbought)

}
```

weight: number; // Peso na pontuação (0-100)

}

1.3 Perfis Padrão Implementados

1. Perfil Seguro (SAFE)

Parâmetro	Valor
Risco	Low
Alavancagem	2x
Capital Inicial	\$100
Margin por Trade	\$20
Threshold de Confiança	80
Take Profit	5%
Stop Loss	2%
Max Drawdown	5%
Ativo por Padrão	Não
Workflow	Trend Check, Low Volatility
Divergências	Não
Padrões de Candle	Não

Lógica: Exige sinal muito forte (80 de confiança), pouca exposição por ordem (\$20), foco em proteger capital com movimentos curtos.

2. Perfil Moderado (MODERATE)

Parâmetro	Valor
Risco	Med
Alavancagem	5x

Parâmetro	Valor
Capital Inicial	\$100
Margin por Trade	\$50
Threshold de Confiança	65
Take Profit	10%
Stop Loss	5%
Max Drawdown	10%
Ativo por Padrão	Sim
Workflow	Trend Follow, RSI Check
Divergências	Sim
Padrões de Candle	Não

Lógica: Confiança média (65), permite mais sinais que Seguro, aumenta alavancagem e tamanho por trade (\$50), usa divergências como filtro extra.

3. Perfil Ousado (BOLD)

Parâmetro	Valor
Risco	High
Alavancagem	10x
Capital Inicial	\$100
Margin por Trade	\$30
Threshold de Confiança	50
Take Profit	20%
Stop Loss	10%
Max Drawdown	20%
Ativo por Padrão	Não

Parâmetro	Valor
Workflow	Breakout, High Volatility
Divergências	Sim
Padrões de Candle	Sim

Lógica: Aceita muito mais sinais (threshold 50), alavancagem alta (10x), focado em breakouts e alta volatilidade, usa divergências e padrões de candle.

4. Perfil Especialista (SPECIALIST)

Parâmetro	Valor
Risco	Expert
Alavancagem	20x
Capital Inicial	\$100
Margin por Trade	\$25
Threshold de Confiança	85
Take Profit	15%
Stop Loss	5%
Max Drawdown	15%
Ativo por Padrão	Não
Workflow	Fibonacci, Order Flow
Divergências	Sim
Padrões de Candle	Sim

Lógica: Agressivo na alavancagem (20x), mas extremamente seletivo (confiança 85), usa conceitos avançados (Fibonacci, order flow), tenta casar alta qualidade de sinal com força de movimento maior.

5. Perfil Alpha Predator (ALPHA)

Parâmetro	Valor
Risco	Extreme
Alavancagem	50x
Capital Inicial	\$100
Margin por Trade	\$20
Threshold de Confiança	50
Take Profit	4%
Stop Loss	2%
Max Drawdown	30%
Ativo por Padrão	Sim
Workflow	HFT Algo, Liquidation Hunt
Divergências	Sim
Padrões de Candle	Sim

Lógica: Altíssimo risco (50x), threshold médio (50), TP/SL curtos (4%/2%), estilo scalping/HFT com muitas operações pequenas.

1.4 Fluxo de Geração de Sinais

O arquivo `technicalAnalysis.ts` implementa a análise técnica unificada:

```
export const unifiedTechnicalAnalysis = (candles: any[], profile: StrategyProfile): AnalysisResult => {
```

```
    // 1. Extraí closes, highs, lows, volumes
```

```
    // 2. Calcula indicadores habilitados no perfil
```

```
    // 3. Gera pontuação de compra/venda baseada em pesos
```

```
    // 4. Retorna sinal (BUY, SELL, NEUTRAL) com confiança
```


}

Indicadores Calculados:

- **RSI (Relative Strength Index):** Detecta sobrevenda/sobrecompra
- **MACD (Moving Average Convergence Divergence):** Detecta cruzamentos de tendência
- **Bollinger Bands:** Detecta rejeição em limites
- **Volume:** Detecta volume climático
- **Stochastic:** Detecta reversão
- **Ichimoku:** Detecta tendência na nuvem
- **SAR (Parabolic):** Detecta flip de tendência
- **CCI:** Detecta ciclos

Pontuação:

- Cada indicador contribui com seus **weight** pontos quando ativo
- Pontuação máxima teórica: ~100 pontos
- Confiança = (scoreBuy ou scoreSell) / pontuação máxima * 100
- Se confiança >= confidenceThreshold do perfil → TRADE

1.5 Integração com Supabase

Tabelas Principais

A aplicação utiliza as seguintes tabelas no Supabase:

Tabela	Descrição	Campos Principais
profiles	Dados de usuário	id, email, created_at, updated_at
strategies	Perfis de investimento	id, user_id, name, leverage, confidence_threshold, active
exchanges	Corretoras conectadas	id, user_id, name, api_key, api_secret, is_testnet
trades	Histórico de trades	id, user_id, symbol, side, entry_price, exit_price, pnl
auth_audit	Log de autenticação	id, user_id, action, timestamp

Proxy Vercel

Para contornar bloqueios de ISP/WAF, todas as requisições ao Supabase são roteadas através de um proxy Vercel:

// Browser → POST /api/supabase (Vercel proxy)

// Vercel proxy → POST Supabase Edge Function

// Supabase Edge Function → Supabase REST API

Isso evita que queries SQL-like (select=*, eq.) sejam bloqueadas por firewalls.

1.6 Integração com Corretoras

Binance Futures

```
export async function callBinanceProxy(  
  
  endpoint: string,  
  
  method: string,  
  
  params: any,  
  
  exchange: Exchange  
  
) {  
  
  // Chama Supabase Edge Function: exchange-proxy  
  
  // Edge Function roteia para Binance API  
  
  // Retorna dados da corretora  
  
}
```

Endpoints Suportados:

- `/fapi/v1/exchangeInfo` - Informações de pares
- `/fapi/v2/account` - Dados da conta
- `/fapi/v1/openOrders` - Ordens abertas
- `/fapi/v1/allOrders` - Histórico de ordens

- `/fapi/v1/order` - Criar/cancelar ordens
- `/fapi/v1/klines` - Candles históricos

Hyperliquid

Suporte adicional para Hyperliquid via `hyperliquidService.ts`:

```
export async function fetchHLAccountData(exchange: Exchange): Promise<RealAccountData>
```

```
export async function validateHLCredentials(apiKey: string, secret: string): Promise<boolean>
```

```
export async function fetchHLTradeHistory(exchange: Exchange): Promise<Trade[]>
```

1.7 Gestão de Risco Atual

Modo Geral (General Risk Mode)

// Usa metas/stops globais para toda a conta

`dailyTargetPct = 10%` // Meta diária: 10% de lucro

`dailyStopLossPct = 5%` // Stop diário: 5% de perda

Modo Perfis (Profile Risk Mode)

// Cada perfil tem suas próprias metas/stops

Seguro: `profileDailyTargetPct = 2%`, `profileDailyStopLossPct = 1%`

Moderado: `profileDailyTargetPct = 5%`, `profileDailyStopLossPct = 3%`

Ousado: `profileDailyTargetPct = 10%`, `profileDailyStopLossPct = 5%`

Especialista: `profileDailyTargetPct = 8%`, `profileDailyStopLossPct = 4%`

Alpha: `profileDailyTargetPct = 15%`, `profileDailyStopLossPct = 8%`

Modo Livre (Free Risk Mode)

// Sem limites diários; motor não pausa automaticamente

2. Arquitetura de Agentes PDCA Independentes

2.1 Visão Geral da Transformação

A proposta transforma cada perfil de investimento em um **agente de IA autônomo** que executa continuamente o ciclo PDCA. Diferentemente da implementação atual (que é reativa e estática), os agentes serão **proativos, adaptativos e competitivos**.

ANTES (Atual):

Perfil Estático	
└ Indicadores fixos	
└ TP/SL fixos	
└ Alavancagem fixa	
└ Sem aprendizado	

↓

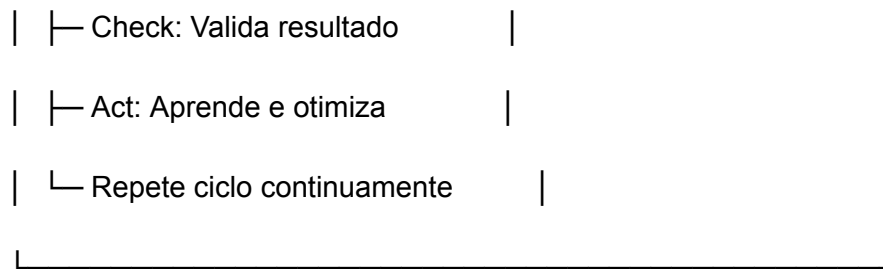
Executa trade

↓

Resultado estático

DEPOIS (Proposto):

Agente Autônomo com PDCA	
└ Plan: Analisa mercado	
└ Do: Executa trade	



↓

Executa trade

↓

Aprende com resultado

↓

Otimiza parâmetros

↓

Próximo trade é melhor

2.2 Arquitetura Detalhada do Agente

2.2.1 Componentes Principais

```
interface AIAgent {
```

```
    // Identidade
```

```
    id: string;           // UUID único
```

```
    profileId: StrategyType; // SAFE, MODERATE, BOLD, SPECIALIST, ALPHA
```

```
    userId: string;       // ID do usuário
```

```
    name: string;         // "Agente Seguro", etc.
```

```
// Estado
```

```
status: 'IDLE' | 'ANALYZING' | 'EXECUTING' | 'LEARNING';
```

```
isEnabled: boolean;           // Usuário pode ativar/desativar
```

```
// Métricas de Desempenho
```

```
metrics: {
```

```
    totalTrades: number;
```

```
    winRate: number;
```

```
    profitFactor: number;
```

```
    sharpeRatio: number;
```

```
    maxDrawdown: number;
```

```
    recoveryFactor: number;
```

```
    avgTradeTime: number;      // Duração média em horas
```

```
    lastTrade: {
```

```
        timestamp: string;
```

```
        symbol: string;
```

```
        pnl: number;
```

```
        confidence: number;
```

```
    };
```

```
};
```

```
// Modelos de IA
```

```
models: {  
  
    lstm: LSTMModel;          // Previsão de preço  
  
    regression: RegressionModel;    // Previsão de volatilidade  
  
    ensemble: EnsembleModel;      // Combinação de modelos  
  
};
```

```
// Histórico de Decisões
```

```
decisionHistory: DecisionRecord[]; // Últimas 1000 decisões
```

```
// Parâmetros Adaptativos
```

```
adaptiveParams: {  
  
    confidenceThreshold: number;    // Ajustado dinamicamente  
  
    leverage: number;              // Ajustado por volatilidade  
  
    marginPerTrade: number;        // Ajustado por capital  
  
    takeProfitMultiplier: number;  // Ajustado por volatilidade  
  
    stopLossMultiplier: number;    // Ajustado por volatilidade  
  
};
```

```
}
```

```
interface DecisionRecord {
```

```
    timestamp: string;
```

```
    symbol: string;
```

```
signal: 'BUY' | 'SELL' | 'HOLD';

confidence: number;

technicalScore: number;

sentimentScore: number;

newsImpact: number;

modelPrediction: number;

actualResult: 'WIN' | 'LOSS' | 'BREAK_EVEN';

pnl: number;

}
```

2.2.2 Módulo PLAN (Planejamento)

```
class PlanModule {

  async analyzeTechnical(symbol: string, candles: Candle[]): Promise<TechnicalAnalysis> {

    // 1. Calcula indicadores técnicos

    const rsi = calculateRSI(candles);

    const macd = calculateMACD(candles);

    const bollinger = calculateBollinger(candles);

    const volume = analyzeVolume(candles);


    // 2. Detecta padrões

    const patterns = detectCandlePatterns(candles);

    const divergences = detectDivergences(candles);
```



```
// 3. Gera score técnico

const technicalScore = this.calculateTechnicalScore({

  rsi, macd, bollinger, volume, patterns, divergences

});

return { technicalScore, details: {...} };

}

async analyzeFundamental(symbol: string): Promise<FundamentalAnalysis> {

  // 1. Busca notícias recentes

  const news = await fetchRecentNews(symbol);

  // 2. Analisa sentimento

  const sentiment = await analyzeSentiment(news);

  // 3. Detecta eventos críticos

  const events = detectCriticalEvents(news);

  // 4. Gera score fundamental

  const fundamentalScore = this.calculateFundamentalScore({

    sentiment, events
```

```
});
```

```
    return { fundamentalScore, sentiment, events };  
}
```

```
async predictPrice(symbol: string, candles: Candle[]): Promise<PricePrediction> {
```

```
    // 1. Prepara dados para LSTM
```

```
    const features = this.prepareFeatures(candles);
```

```
    // 2. Executa modelo LSTM
```

```
    const lstmPrediction = await this.lstmModel.predict(features);
```

```
    // 3. Executa modelo de regressão para volatilidade
```

```
    const volatilityPrediction = await this.regressionModel.predict(features);
```

```
    // 4. Ensemble: combina previsões
```

```
    const ensemblePrediction = this.ensembleModel.combine([
```

```
        lstmPrediction,
```

```
        volatilityPrediction
```

```
    ]);
```

```
    return {
```

```
    priceTarget: ensemblePrediction.price,  
  
    volatilityForecast: ensemblePrediction.volatility,  
  
    confidence: ensemblePrediction.confidence  
};  
}
```

```
async generateSignal(symbol: string): Promise<TradingSignal> {  
  
    // 1. Análise técnica  
  
    const technical = await this.analyzeTechnical(symbol, candles);  
  
  
    // 2. Análise fundamental  
  
    const fundamental = await this.analyzeFundamental(symbol);  
  
  
    // 3. Previsão de preço  
  
    const prediction = await this.predictPrice(symbol, candles);  
  
  
    // 4. Combina tudo em sinal único  
  
    const signal = this.combineSignals({  
  
        technical: technical.score,  
  
        fundamental: fundamental.score,  
  
        prediction: prediction.confidence
```

```
});
```

```
return signal;
```

```
}
```

```
}
```

2.2.3 Módulo DO (Execução)

```
class DoModule {
```

```
  async calculatePositionSize(
```

```
    capital: number,
```

```
    riskPercentage: number,
```

```
    stopLossDistance: number
```

```
  ): Promise<number> {
```

```
    // Kelly Criterion para sizing ótimo
```

```
    const winRate = this.agent.metrics.winRate;
```

```
    const profitFactor = this.agent.metrics.profitFactor;
```

```
    //  $f^* = (bp - q) / b$ 
```

```
    // onde: b = razão risco/retorno, p = win rate, q = 1 - p
```

```
    const kellyFraction = ((profitFactor * winRate) - (1 - winRate)) / profitFactor;
```

```
    // Aplica fração conservadora (25% do Kelly para evitar ruína)
```

```
    const conservativeFraction = Math.max(0.01, kellyFraction * 0.25);
```

```

// Calcula tamanho da posição

const positionSize = (capital * riskPercentage * conservativeFraction) / stopLossDistance;

return Math.min(positionSize, capital * 0.1); // Máximo 10% do capital por trade
}

```

```

async calculateDynamicTP_SL(
  entryPrice: number,
  volatility: number,
  signal: TradingSignal
): Promise<{ tp: number; sl: number }> {
  // VPM: Volatility Price Movement

  const atr = volatility;

  // Ajusta TP/SL baseado em volatilidade

  const tpMultiplier = 1.5 + (volatility / 100); // Mais volatilidade = TP maior

  const slMultiplier = 1.0 - (volatility / 200); // Mais volatilidade = SL maior

  const tp = entryPrice + (atr * tpMultiplier);

  const sl = entryPrice - (atr * slMultiplier);

```

```
    return { tp, sl };  
}
```

```
async executeOrder(  
    symbol: string,  
    signal: TradingSignal,  
    prediction: PricePrediction  
): Promise<ExecutionResult> {  
    try {  
        // 1. Calcula tamanho da posição  
        const positionSize = await this.calculatePositionSize(  
            this.agent.capital,  
            0.02, // 2% risco por trade  
            signal.stopLoss  
        );  
  
        // 2. Calcula TP/SL dinâmico  
        const { tp, sl } = await this.calculateDynamicTP_SL(  
            signal.entryPrice,  
            prediction.volatilityForecast,  
            signal
```

```
);
```

```
// 3. Executa ordem na corretora
```

```
const order = await executeOrderOnExchange({  
  symbol,  
  side: signal.direction,  
  quantity: positionSize,  
  leverage: this.agent.adaptiveParams.leverage,  
  stopLossPrice: sl,  
  takeProfitPrice: tp  
});
```

```
// 4. Registra execução
```

```
this.recordExecution({  
  timestamp: new Date().toISOString(),  
  symbol,  
  signal,  
  positionSize,  
  tp,  
  sl,  
  orderId: order.id
```

```

});

return { success: true, orderId: order.id };
} catch (error) {

    console.error('Execution error:', error);

    return { success: false, error: error.message };
}
}

async monitorPosition(orderId: string): Promise<PositionStatus> {

    // Monitora posição em tempo real

    // Detecta: TP atingido, SL atingido, erro de API, etc.

    const position = await fetchPositionStatus(orderId);

    if (position.status === 'CLOSED') {

        return {

            status: 'CLOSED',

            pnl: position.pnl,

            reason: position.closeReason

        };
    }
}

```



```
    return { status: 'OPEN', currentPnL: position.unrealizedPnL };  
  }  
}
```

2.2.4 Módulo CHECK (Verificação)

```
class CheckModule {  
  
  async validateExecution(orderId: string): Promise<ExecutionValidation> {  
  
    // 1. Verifica se ordem foi executada na blockchain  
  
    const onChainOrder = await fetchOrderFromBlockchain(orderId);  
  
    if (!onChainOrder) {  
  
      return {  
  
        valid: false,  
  
        error: 'Ordem não encontrada na blockchain'  
  
      };  
    }  
  
    // 2. Reconcilia com saldo da carteira  
  
    const walletBalance = await fetchWalletBalance();  
  
    const expectedBalance = this.previousBalance - onChainOrder.cost;
```

```
if (Math.abs(walletBalance - expectedBalance) > 0.01) {  
  
    return {  
  
        valid: false,  
  
        error: 'Discrepância de saldo detectada'  
  
    };  
  
}  
  
return { valid: true };  
}  
  
async calculateMetrics(trade: ClosedTrade): Promise<TradeMetrics> {  
  
    const entryPrice = trade.entryPrice;  
  
    const exitPrice = trade.exitPrice;  
  
    const quantity = trade.quantity;  
  
    const leverage = trade.leverage;  
  
    const entryTime = new Date(trade.entryTime);  
  
    const exitTime = new Date(trade.exitTime);  
  
  
    // 1. PnL  
  
    const pnlPercent = ((exitPrice - entryPrice) / entryPrice) * 100;  
  
    const pnlUSD = (exitPrice - entryPrice) * quantity * leverage;
```

```
// 2. Razão Risco/Retorno
```

```
const riskRewardRatio = Math.abs(pnlPercent / trade.stopLossPercent);
```

```
// 3. Duração
```

```
const durationHours = (exitTime.getTime() - entryTime.getTime()) / (1000 * 60 * 60);
```

```
// 4. Comparação com Benchmark
```

```
const benchmarkReturn = this.calculateBenchmarkReturn(trade.symbol, entryTime, exitTime);
```

```
const outperformance = pnlPercent - benchmarkReturn;
```

```
return {
```

```
  pnlPercent,
```

```
  pnlUSD,
```

```
  riskRewardRatio,
```

```
  durationHours,
```

```
  outperformance,
```

```
  wasWin: pnlUSD > 0
```

```
};
```

```
}
```

```
async analyzeContext(trade: ClosedTrade): Promise<TradeContext> {
```

```
// Analisa contexto de mercado durante a operação
```

```
const volatility = await calculateVolatilityDuring(  
    trade.symbol,  
    trade.entryTime,  
    trade.exitTime  
);
```

```
const correlation = await calculateCorrelationWithBTC(  
    trade.symbol,  
    trade.entryTime,  
    trade.exitTime  
);
```

```
const newsEvents = await fetchNewsEventsDuring(  
    trade.symbol,  
    trade.entryTime,  
    trade.exitTime  
);
```

```
return {
```

```

    volatility,

    correlation,

    newsEvents,

    marketCondition: this.classifyMarketCondition(volatility, correlation)

  };
}

```

```

async generateTradeReport(trade: ClosedTrade): Promise<TradeReport> {

  const metrics = await this.calculateMetrics(trade);

  const context = await this.analyzeContext(trade);

  return {

    trade,

    metrics,

    context,

    timestamp: new Date().toISOString()

  };

}

```

2.2.5 Módulo ACT (Ação/Aprendizado)

```

class ActModule {

  async retrainModels(trades: ClosedTrade[]): Promise<void> {

```

// 1. Prepara dados de treinamento

```
const trainingData = this.prepareTrainingData(trades);
```

// 2. Valida com cross-validation

```
const { trainSet, testSet } = this.splitData(trainingData, 0.8);
```

// 3. Retreina LSTM

```
console.log('Retraining LSTM model...');
```

```
const lstmMetrics = await this.agent.models.lstm.train(trainSet);
```

```
const lstmValidation = await this.agent.models.lstm.validate(testSet);
```

```
if (lstmValidation.accuracy < 0.45) {
```

```
    console.warn('LSTM accuracy below threshold, keeping previous model');
```

```
} else {
```

```
    console.log('LSTM model updated successfully');
```

```
}
```

// 4. Retreina Regression

```
console.log('Retraining Regression model...');
```

```
const regressionMetrics = await this.agent.models.regression.train(trainSet);
```

```
const regressionValidation = await this.agent.models.regression.validate(testSet);
```

```
// 5. Atualiza Ensemble

this.agent.models.ensemble.updateWeights([

  lstmValidation.accuracy,

  regressionValidation.accuracy

]);

}

async optimizeParameters(): Promise<void> {

  // Algoritmo genético para otimizar parâmetros


  const population = this.generateInitialPopulation(50);


  for (let generation = 0; generation < 10; generation++) {

    // 1. Avalia fitness de cada indivíduo

    const fitness = await Promise.all(

      population.map(individual => this.evaluateFitness(individual))

    );


    // 2. Seleciona melhores (top 20%)

    const topIndividuals = this.selectTop(population, fitness, 10);
```

```
// 3. Crossover e mutação
```

```
const offspring = this.createOffspring(topIndividuals);
```

```
// 4. Nova população
```

```
population.splice(0, population.length, ...topIndividuals, ...offspring);
```

```
console.log(`Generation ${generation}: Best fitness = ${Math.max(...fitness)}`);
```

```
}
```

```
// Aplica melhores parâmetros
```

```
const bestIndividual = population[0];
```

```
this.agent.adaptiveParams = bestIndividual.params;
```

```
}
```

```
async detectPatterns(): Promise<void> {
```

```
// Detecta padrões bem-sucedidos no histórico
```

```
const winningTrades = this.agent.decisionHistory
```

```
.filter(d => d.actualResult === 'WIN')
```

```
.slice(-100);
```



```
// Agrupa por características
```

```
const patterns = new Map<string, number>();
```

```
for (const trade of winningTrades) {
```

```
    const pattern = this.extractPattern(trade);
```

```
    patterns.set(pattern, (patterns.get(pattern) || 0) + 1);
```

```
}
```

```
// Ordena por frequência
```

```
const sortedPatterns = Array.from(patterns.entries())
```

```
    .sort((a, b) => b[1] - a[1])
```

```
    .slice(0, 5);
```

```
console.log('Top winning patterns:', sortedPatterns);
```

```
// Aumenta weight desses padrões nos indicadores
```

```
for (const [pattern, count] of sortedPatterns) {
```

```
    this.boostPatternWeight(pattern, count);
```

```
}
```

```
}
```

```
async adjustConfidenceThreshold(): Promise<void> {
```

```
// Ajusta threshold baseado em win rate

const recentWinRate = this.calculateRecentWinRate(50);

if (recentWinRate > 0.60) {

    // Win rate alto: pode ser mais agressivo

    this.agent.adaptiveParams.confidenceThreshold = Math.max(

        40,

        this.agent.adaptiveParams.confidenceThreshold - 5

    );

} else if (recentWinRate < 0.40) {

    // Win rate baixo: precisa ser mais seletivo

    this.agent.adaptiveParams.confidenceThreshold = Math.min(

        95,

        this.agent.adaptiveParams.confidenceThreshold + 5

    );

}

}

async adjustLeverage(): Promise<void> {

    // Ajusta leverage baseado em volatilidade e drawdown
```

```
const currentDrawdown = this.calculateCurrentDrawdown();

const marketVolatility = await this.fetchMarketVolatility();


if (currentDrawdown > this.agent.maxDrawdown * 0.8) {

    // Perto do limite de drawdown: reduz leverage

    this.agent.adaptiveParams.leverage = Math.max(

        1,

        this.agent.adaptiveParams.leverage * 0.8

    );

} else if (marketVolatility < 20 && currentDrawdown < this.agent.maxDrawdown * 0.3) {

    // Mercado calmo e drawdown baixo: pode aumentar leverage

    this.agent.adaptiveParams.leverage = Math.min(

        this.agent.leverage * 1.2,

        this.agent.adaptiveParams.leverage * 1.1

    );

}

}

async storeKnowledge(): Promise<void> {

    // Armazena conhecimento em banco de dados
```

```

const knowledge = {

  agentId: this.agent.id,

  timestamp: new Date().toISOString(),

  successfulPatterns: this.extractSuccessfulPatterns(),

  failedPatterns: this.extractFailedPatterns(),

  optimizedParameters: this.agent.adaptiveParams,

  metrics: this.agent.metrics,

  modelWeights: this.agent.models.ensemble.weights

};

await saveKnowledgeToDatabase(knowledge);

}

}

```

2.3 Ciclo PDCA Diário

08:00 UTC - INICIALIZAÇÃO

- └ Carrega modelos treinados do dia anterior
- └ Carrega parâmetros otimizados
- └ Inicializa estado do agente

08:00 - 20:00 UTC - OPERAÇÃO CONTÍNUA (A cada 5 minutos)

- └ PLAN (5 min)
- | └ Analisa técnica

- | └─ Analisa fundamental (notícias)
- | └─ Prevê preço com LSTM
- | └─ Gera sinal de trading
- └─ DO (5 min)
- | └─ Se sinal confiável: executa trade
- | └─ Calcula TP/SL dinâmico (VPM)
- | └─ Monitora posição
- └─ CHECK (contínuo)
- | └─ Valida execução
- | └─ Monitora PnL
- | └─ Detecta eventos críticos
- └─ Aguarda próximo ciclo

20:00 - 22:00 UTC - CONSOLIDAÇÃO

- └─ ACT (Aprendizado)
- | └─ Retreina modelos LSTM/Regression
- | └─ Otimiza parâmetros (algoritmo genético)
- | └─ Detecta padrões bem-sucedidos
- | └─ Ajusta confidence threshold
- | └─ Ajusta leverage
- | └─ Armazena conhecimento
- └─ Prepara para próximo dia

22:00 - 08:00 UTC - REPOUSO

└─ Monitora posições abertas passivamente

└─ Aguarda próximo ciclo

└─ Mantém alertas críticos ativos

2.4 Sistema de Competição Entre Agentes

2.4.1 Leaderboard Diário

```
interface AgentLeaderboard {
```

```
    date: string;
```

```
    agents: AgentScore[];
```

```
}
```

```
interface AgentScore {
```

```
    agentId: string;
```

```
    profileName: string;
```

```
    rank: number;
```

```
    pnlUSD: number;
```

```
    pnlPercent: number;
```

```
    winRate: number;
```

```
    sharpeRatio: number;
```

```
    trades: number;
```

```
    maxDrawdown: number;
```

```
    recoveryFactor: number;
```

```
}
```

2.4.2 Competição Semanal e Mensal

SEMANAL:

- |— Agente com maior PnL: +10% bonus ao capital
- |— Agente com maior Sharpe Ratio: +5% bonus
- |— Agente com maior Win Rate: +5% bonus
- └— Agente com melhor Recovery Factor: +5% bonus

MENSAL:

- |— Top 3 agentes: Recebem 20%, 15%, 10% bonus
- |— Agente com melhor consistência: +15% bonus
- └— Agente com melhor aprendizado: +10% bonus

2.4.3 Mecanismo de Recompensa

```
interface AgentReward {
```

```
    agentId: string;
```

```
    period: 'daily' | 'weekly' | 'monthly';
```

```
    baseReward: number;
```

```
    performanceBonus: number;
```

```
    consistencyBonus: number;
```

```
    learningBonus: number;
```

```
    totalReward: number;
```

```
    appliedCapital: number;
```

```
}
```

3. Banco de Dados Supabase - Estrutura Proposta

3.1 Tabelas Existentes

Tabela: profiles

```
CREATE TABLE profiles (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  email TEXT UNIQUE NOT NULL,  
  
  full_name TEXT,  
  
  created_at TIMESTAMP DEFAULT now(),  
  
  updated_at TIMESTAMP DEFAULT now()  
  
);
```

Tabela: strategies

```
CREATE TABLE strategies (  
  
  id TEXT PRIMARY KEY,  
  
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,  
  
  name TEXT NOT NULL,  
  
  description TEXT,  
  
  risk_level TEXT,  
  
  confidence_threshold NUMERIC,  
  
  leverage NUMERIC,  
  
  capital NUMERIC,  
  
  current_capital NUMERIC,
```



```
allocated_capital NUMERIC,  
  
margin_per_trade NUMERIC,  
  
pnl NUMERIC,  
  
trades INT,  
  
win_rate NUMERIC,  
  
active BOOLEAN DEFAULT false,  
  
stop_loss NUMERIC,  
  
take_profit NUMERIC,  
  
max_drawdown NUMERIC,  
  
workflow_steps TEXT[],  
  
indicators JSONB,  
  
use_divergences BOOLEAN,  
  
use_candle_patterns BOOLEAN,  
  
created_at TIMESTAMP DEFAULT now(),  
  
updated_at TIMESTAMP DEFAULT now()  
  
);
```

Tabela: exchanges

```
CREATE TABLE exchanges (  
  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,  
  
name TEXT,  
  
type TEXT,
```

```
api_key TEXT,  
  
api_secret TEXT,  
  
is_testnet BOOLEAN,  
  
status TEXT,  
  
created_at TIMESTAMP DEFAULT now()  
  
);
```

Tabela: trades

```
CREATE TABLE trades (  
  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,  
  
symbol TEXT NOT NULL,  
  
side TEXT,  
  
entry_price NUMERIC,  
  
exit_price NUMERIC,  
  
amount NUMERIC,  
  
pnl NUMERIC,  
  
status TEXT,  
  
timestamp TIMESTAMP DEFAULT now(),  
  
strategy_id TEXT,  
  
strategy_name TEXT  
  
);
```

3.2 Novas Tabelas para Agentes PDCA

Tabela: ai_agents

```
CREATE TABLE ai_agents (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,  
  
  profile_id TEXT NOT NULL,  
  
  name TEXT NOT NULL,  
  
  description TEXT,  
  
  status TEXT DEFAULT 'IDLE',  
  
  is_enabled BOOLEAN DEFAULT false,  
  
  
  -- Métricas  
  
  total_trades INT DEFAULT 0,  
  
  win_rate NUMERIC DEFAULT 0,  
  
  profit_factor NUMERIC DEFAULT 0,  
  
  sharpe_ratio NUMERIC DEFAULT 0,  
  
  max_drawdown NUMERIC DEFAULT 0,  
  
  recovery_factor NUMERIC DEFAULT 0,  
  
  avg_trade_time NUMERIC DEFAULT 0,  
  
  
  -- Parâmetros Adaptativos
```

confidence_threshold NUMERIC,

leverage NUMERIC,

margin_per_trade NUMERIC,

tp_multiplier NUMERIC,

sl_multiplier NUMERIC,

-- Modelos

lstm_model BYTEA,

regression_model BYTEA,

ensemble_weights JSONB,

created_at TIMESTAMP DEFAULT now(),

updated_at TIMESTAMP DEFAULT now(),

last_training TIMESTAMP,

UNIQUE(user_id, profile_id)

);

Tabela: agent_decisions

CREATE TABLE agent_decisions (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

agent_id UUID REFERENCES ai_agents(id) ON DELETE CASCADE,

timestamp TIMESTAMP DEFAULT now(),

symbol TEXT NOT NULL,

signal TEXT,

confidence NUMERIC,

technical_score NUMERIC,

sentiment_score NUMERIC,

news_impact NUMERIC,

model_prediction NUMERIC,

actual_result TEXT,

pnl NUMERIC,

INDEX idx_agent_timestamp (agent_id, timestamp DESC)

);

Tabela: agent_trades

CREATE TABLE agent_trades (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

agent_id UUID REFERENCES ai_agents(id) ON DELETE CASCADE,

symbol TEXT NOT NULL,

side TEXT,

entry_price NUMERIC,

entry_time TIMESTAMP,

exit_price NUMERIC,

exit_time TIMESTAMP,

quantity NUMERIC,

pnl NUMERIC,

pnl_percent NUMERIC,

tp_level NUMERIC,

sl_level NUMERIC,

leverage NUMERIC,

status TEXT,

INDEX idx_agent_status (agent_id, status)

);

Tabela: agent_knowledge

CREATE TABLE agent_knowledge (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

agent_id UUID REFERENCES ai_agents(id) ON DELETE CASCADE,

date DATE DEFAULT CURRENT_DATE,

-- Padrões

successful_patterns JSONB,

failed_patterns JSONB,

-- Parâmetros Otimizados

optimized_parameters JSONB,

-- Métricas

daily_metrics JSONB,

-- Pesos dos Modelos

model_weights JSONB,

created_at TIMESTAMP DEFAULT now(),

UNIQUE(agent_id, date)

);

Tabela: agent_leaderboard

CREATE TABLE agent_leaderboard (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,

date DATE DEFAULT CURRENT_DATE,

period TEXT, -- 'daily', 'weekly', 'monthly'

agent_id UUID REFERENCES ai_agents(id),

profile_name TEXT,

```
rank INT,  
  
pnl_usd NUMERIC,  
  
pnl_percent NUMERIC,  
  
win_rate NUMERIC,  
  
sharpe_ratio NUMERIC,  
  
trades INT,  
  
max_drawdown NUMERIC,  
  
recovery_factor NUMERIC,  
  
  
created_at TIMESTAMP DEFAULT now(),  
  
  
INDEX idx_user_date (user_id, date DESC)  
  
);
```

Tabela: agent_rewards

```
CREATE TABLE agent_rewards (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  agent_id UUID REFERENCES ai_agents(id) ON DELETE CASCADE,  
  
  period TEXT, -- 'daily', 'weekly', 'monthly'  
  
  date DATE,  
  
  
  base_reward NUMERIC,  
  
  performance_bonus NUMERIC,
```



```
consistency_bonus NUMERIC,  
  
learning_bonus NUMERIC,  
  
total_reward NUMERIC,  
  
applied_capital NUMERIC,  
  
  
created_at TIMESTAMP DEFAULT now()  
  
);
```

Tabela: news_feeds

```
CREATE TABLE news_feeds (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  title TEXT NOT NULL,  
  
  source TEXT,  
  
  url TEXT,  
  
  published_at TIMESTAMP,  
  
  fetched_at TIMESTAMP DEFAULT now(),  
  
  sentiment TEXT, -- 'POSITIVE', 'NEGATIVE', 'NEUTRAL'  
  
  confidence NUMERIC,  
  
  impact_level TEXT, -- 'HIGH', 'MEDIUM', 'LOW'  
  
  related_symbols TEXT[],  
  
  raw_content TEXT,
```

INDEX idx_published (published_at DESC),

INDEX idx_symbols (related_symbols)

);

4. Implementação Prática: Roadmap Detalhado

Fase 1: Infraestrutura (Semanas 1-4)

Semana 1-2: Setup de Agentes

- ☐ Criar tabelas Supabase para agentes
- ☐ Implementar classe base **AIAgent**
- ☐ Criar sistema de persistência de agentes
- ☐ Implementar logging e monitoramento

Semana 3-4: Módulos Básicos

- ☐ Implementar **PlanModule** (análise técnica básica)
- ☐ Implementar **DoModule** (execução de ordens)
- ☐ Implementar **CheckModule** (validação)
- ☐ Testes unitários

Fase 2: Aprendizado (Semanas 5-8)

Semana 5-6: Modelos de IA

- ☐ Integrar TensorFlow.js para LSTM
- ☐ Treinar modelo de previsão de preço
- ☐ Implementar modelo de regressão para volatilidade
- ☐ Criar ensemble de modelos

Semana 7-8: Módulo ACT

- ☐ Implementar retraining automático
- ☐ Implementar algoritmo genético para otimização
- ☐ Implementar detecção de padrões
- ☐ Testes de convergência

Fase 3: Notícias e Sentimento (Semanas 9-12)

Semana 9-10: Integração de APIs

- ☐ Conectar NewsData.io API
- ☐ Conectar CoinDesk API
- ☐ Implementar agregador de notícias
- ☐ Criar tabela de notícias no Supabase

Semana 11-12: Análise de Sentimento

- ☐ Integrar modelo NLP para sentimento
- ☐ Treinar modelo com dados históricos
- ☐ Integrar sentimento nos sinais de trading
- ☐ Testes de acurácia

Fase 4: Competição (Semanas 13-16)

Semana 13-14: Sistema de Leaderboard

- ☐ Implementar cálculo de rankings
- ☐ Criar tabelas de leaderboard
- ☐ Implementar UI de leaderboard
- ☐ Testes de integridade

Semana 15-16: Sistema de Recompensas

- ☐ Implementar cálculo de recompensas
- ☐ Implementar aplicação de bonus ao capital
- ☐ Criar histórico de recompensas
- ☐ Testes de distribuição

Fase 5: VPM e Mainnet (Semanas 17-20)

Semana 17-18: Implementação VPM

- ☐ Implementar cálculo de ATR dinâmico
- ☐ Implementar ajuste de TP/SL por volatilidade
- ☐ Testes em Testnet
- ☐ Validação de cálculos

Semana 19-20: Preparação para Mainnet

- ☐ Auditoria de segurança
 - ☐ Testes de stress
 - ☐ Documentação operacional
 - ☐ Treinamento de equipe
-

5. Opções de Configuração para Criar Novos Perfis

5.1 Interface de Criação de Perfil

Quando o usuário cria um novo perfil, ele terá as seguintes opções:

```
interface ProfileCreationForm {
```

```
    // Informações Básicas
```

```
    name: string;           // Nome do perfil
```

```
    description: string;    // Descrição
```

```
    icon: string;           // Ícone (shield, scale, rocket, etc.)
```

```
    color: string;         // Cor para UI
```

```
    // Configuração de Risco
```

```
    riskLevel: 'Low' | 'Med' | 'High' | 'Expert' | 'Extreme';
```

```
    leverage: number;       // 1-100x
```

```
    capital: number;        // Capital inicial
```

```
    marginPerTrade: number; // USD por trade
```

```
    maxDrawdown: number;   // % máximo de queda
```

// Filtros de Entrada

confidenceThreshold: number; // 0-100

// Indicadores

indicators: {

 rsi: { enabled: boolean; period: number; thresholdLow: number; thresholdHigh: number;
weight: number };

 macd: { enabled: boolean; weight: number };

 stochastic: { enabled: boolean; weight: number };

 bollinger: { enabled: boolean; weight: number };

 ichimoku: { enabled: boolean; weight: number };

 sar: { enabled: boolean; weight: number };

 cci: { enabled: boolean; weight: number };

 volume: { enabled: boolean; weight: number };

};

// Análise Avançada

useDivergences: boolean;

useCandlePatterns: boolean;

// Workflow

workflowSteps: string[]; // Etapas do pipeline

```

// TP/SL

takeProfit: number;           // %

stopLoss: number;            // %


// Gestão de Risco por Perfil

profileDailyTargetPct?: number; // Meta diária

profileDailyStopLossPct?: number; // Stop diário


// Agente PDCA

enablePDCA: boolean;          // Ativar ciclo PDCA?

enableCompetition: boolean;    // Participar de competição?
}

```

5.2 Presets Sugeridos

Além dos 5 perfis padrão, o sistema oferecerá presets para casos comuns:

Preset	Descrição	Recomendado Para
Iniciante Conservador	2x leverage, 80% confidence	Novo usuário, sem experiência
Iniciante Moderado	5x leverage, 65% confidence	Novo usuário, quer testar
Trader Experiente	10x leverage, 50% confidence	Trader com experiência
Scalper	50x leverage, 4%/2% TP/SL	Operações de alta frequência
Swing Trader	5x leverage, 20%/5% TP/SL	Operações de médio prazo

Preset	Descrição	Recomendado Para
Institucional	20x leverage, 85% confidence	Operações profissionais

6. Recomendações de Implementação

6.1 Prioridades

P1 - Crítico:

1. Migrar para Binance Mainnet com VPM
2. Implementar ciclo PDCA básico
3. Integrar feeds de notícias

P2 - Importante:

1. Sistema de competição entre agentes
2. Análise de sentimento avançada
3. Backtesting formal

P3 - Desejável:

1. Suporte a múltiplas corretoras
2. Gamificação
3. Mobile app

6.2 Tecnologias Recomendadas

Componente	Tecnologia	Razão
LSTM	TensorFlow.js	Executa no browser, sem backend
Ensemble	scikit-learn (Node.js)	Combinação robusta de modelos
Algoritmo Genético	DEAP (Python) ou GA.js	Otimização de parâmetros
NLP Sentimento	Hugging Face Transformers	Modelos pré-treinados

Componente	Tecnologia	Razão
News API	NewsData.io	Cobertura ampla, preço acessível
Backtesting	Backtrader (Python)	Framework robusto

6.3 Métricas de Sucesso

Após 3 meses de implementação:

Métrica	Baseline	Meta
Win Rate	21%	55-60%
Profit Factor	0.74	1.8-2.2
Sharpe Ratio	0.3	1.2-1.5
Max Drawdown	45%	15-20%
PNL Mensal	-\$855	+\$2,000-3,000
Tempo de Resposta	N/A	< 100ms
Uptime	N/A	> 99.5%

Conclusão

A transformação do CAP.PRO em uma plataforma de agentes de IA com ciclo PDCA representa uma evolução fundamental. Ao combinar análise técnica sofisticada, aprendizado contínuo, análise de sentimento e competição entre agentes, a plataforma passará de um executor de estratégias estáticas para um sistema verdadeiramente inteligente e adaptativo.

A implementação proposta é ambiciosa mas viável, com um roadmap claro de 20 semanas para alcançar todas as funcionalidades. O sucesso dependerá de atenção aos detalhes, testes rigorosos e iteração contínua baseada em feedback real de usuários.

Próximos Passos:

1. Validar arquitetura com stakeholders
2. Iniciar desenvolvimento da Fase 1

3. Configurar ambiente de desenvolvimento
4. Recrutar especialistas em ML/IA
5. Estabelecer métricas de sucesso

Fim do Complemento

Este documento complementa o relatório anterior com análise técnica profunda do código existente e arquitetura detalhada para implementação de agentes de IA com ciclo PDCA.